

# Transformer の勉強

June 15, 2026

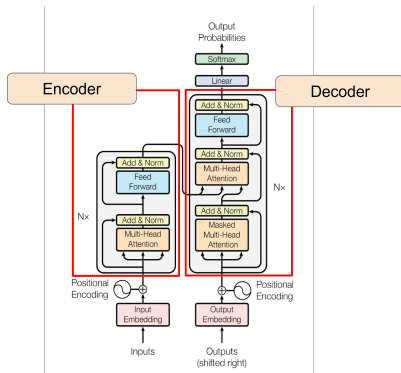
# 目次

---

1. **Transformer** のモデル構造
2. **Attention** の応用
3. 計算過程の解釈・解析について
4. 回路研究の進捗

# モデル構造

## Transformer

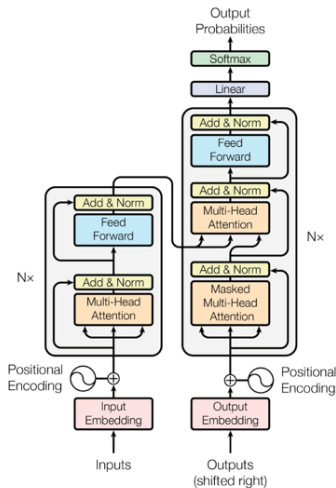


- 左図は, Transformer を構成する最小単位である”ブロック”と呼ばれるもの.
- 縦に  $N$  層積み重ねてつくる.
- 左側 : Encoder ブロック
- 右側 : Decoder ブロック

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# モデル構造

## Transformer



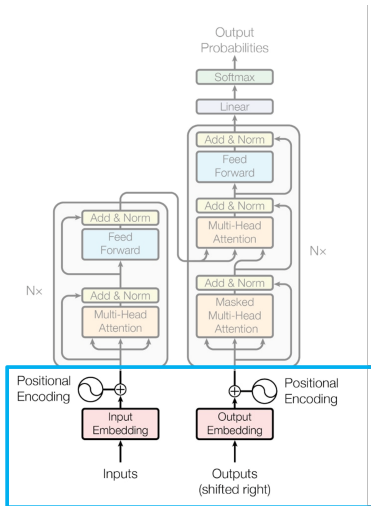
- Embedding
- Multi-Head Attention
- Feed Forward
- Others

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.



# モデル構造 - Embedding

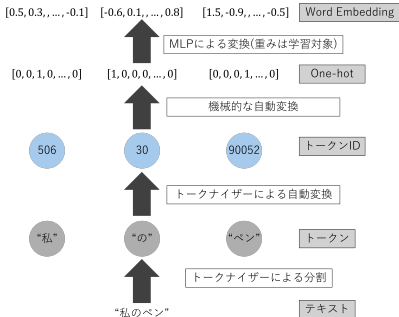
## Transformer



- Embedding
- Multi-Head Attention
- Feed Forward
- Others

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Word Embedding (WE)



総トークン数を  $V$  として、 $V$  次元の One-hot ベクトルを埋め込みに変換する行列  $E$  を定義

$$E \in \mathbb{R}^{V \times d_{\text{model}}}$$

$t=[0,0,\dots,1,\dots,0,0]$  を、第  $k$  成分のみ 1 で、他は全て 0 の One-hot ベクトルとする

$$tE = d$$

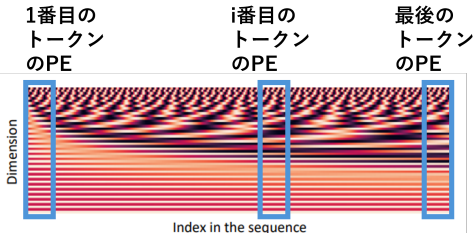
$d$  は  $E$  の  $k$  行目と一致する横ベクトル

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Positional Encoding(PE)

$$p_i = \begin{pmatrix} \sin(i/10000^{2\cdot 1/d}) \\ \cos(i/10000^{2\cdot 1/d}) \\ \vdots \\ \sin(i/10000^{2\cdot \frac{d}{2}/d}) \\ \cos(i/10000^{2\cdot \frac{d}{2}/d}) \end{pmatrix}$$

- $p_i$  :  $i$  番目のトークンの PE
- $d$  : 特徴量  $d_{model}$  の次元数



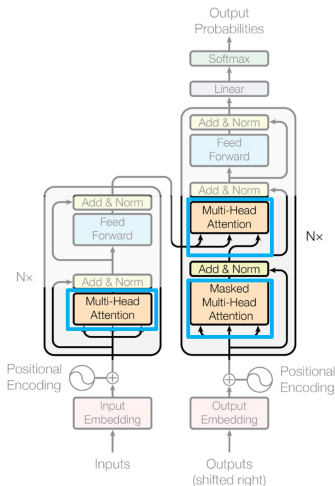
Transformer ブロックは単語の位置情報を把握できないため、**Word Embedding に位置情報 (PE) を加算**.

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

John Hewitt, Natural Language Processing with Deep Learning CS224N/Ling284

# モデル構造 - Multi-head Attention

## Transformer

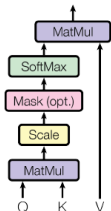


- Embedding
- Multi-Head Attention
- Feed Forward
- Others

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Scaled Dot-Product Attention

Scaled Dot-Product Attention



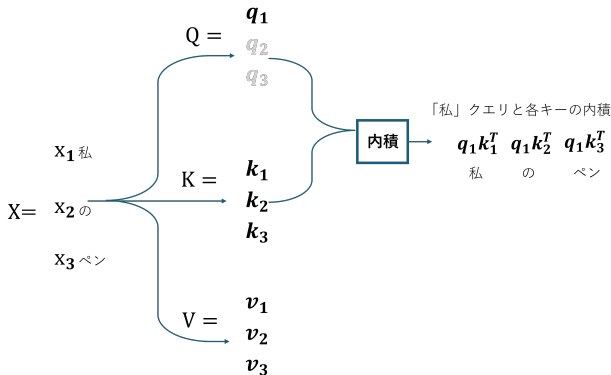
$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

処理フロー

- 1. **MatMul**:  $Q$  と  $K$  の内積により単語間の相関（スコア）を算出
- 2. **Scale**:  $\sqrt{d_k}$  で除算。 $d_k$  増大による勾配消失を防止
- 3. **Softmax**: スコアを合計 1 の確率（重み）に正規化
- 4. **MatMul**: 重みに基づき  $V$  を足し合わせ、文脈を集約

| 変数                                | 名称     | イメージ・定義          |
|-----------------------------------|--------|------------------|
| $Q \in \mathbb{R}^{n \times d_k}$ | Query  | 「何に注目したいか」       |
| $K \in \mathbb{R}^{n \times d_k}$ | Key    | 「どのような情報を持っているか」 |
| $V \in \mathbb{R}^{n \times d_v}$ | Value  | 実際に抽出される「情報」     |
| $n$                               | シーケンス長 | 入力トークン数          |
| $d_k$                             | 次元数    | $Q, K$ の次元数      |
| $d_v$                             | 次元数    | $V$ の次元数         |
| softmax                           | 関数     | 正規化処理            |

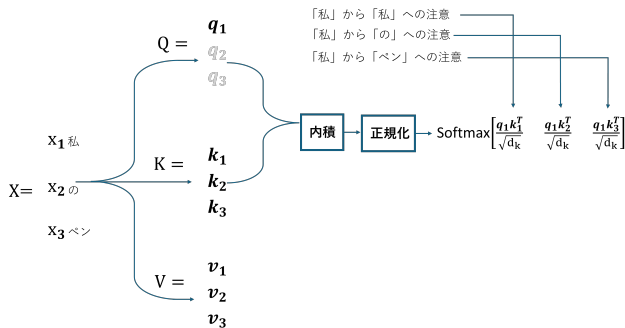
# Scaled Dot-Product Attention 1



Query(注目したい単語) と各 Key(文中の各単語) がどれだけ関連しているか (類似度) を算出

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

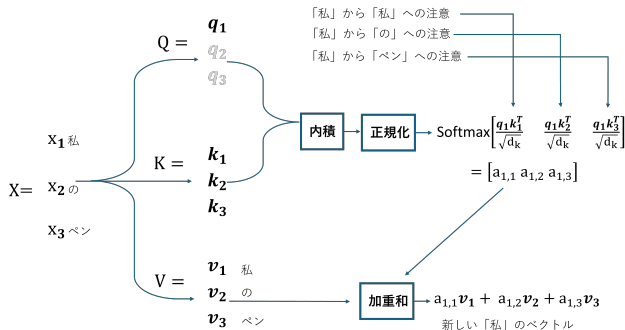
# Scaled Dot-Product Attention 2



Query と各 Key の内積を正規化したものを「対象の単語が周りの各単語をどれほど注視しているか」の指標とみなす (=注意重み)

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Scaled Dot-Product Attention 3

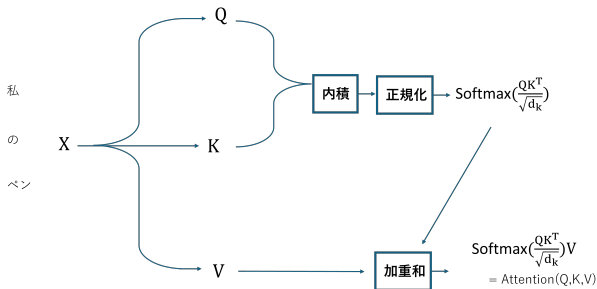


注意重みで重み付けされた Value の和を計算することで「私」のベクトルに周辺の単語情報が付加される

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.



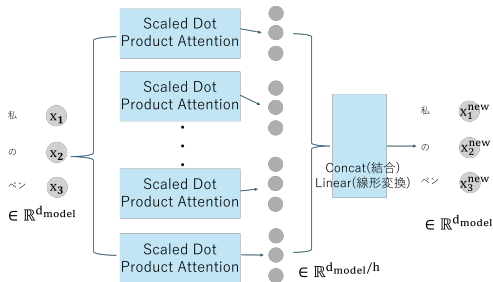
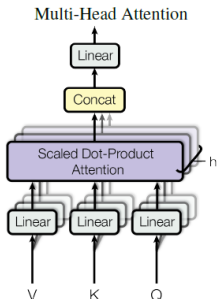
# Scaled Dot-Product Attention 4



$x_1$ 、 $x_2$ 、 $x_3$  の更新は独立しているため並列化可能  
→  $x_1$  を行列  $X$  の 1 列目の要素のようにみなすと  $X$  全体の更新を  
 $\text{Attention}(Q, K, V)$  で行える

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Multi-Head Attention



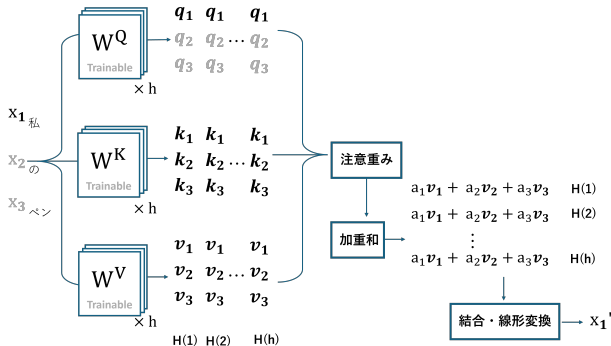
$$\text{Multi Head}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

| 変数                                               | 名称           |
|--------------------------------------------------|--------------|
| $W^Q, W^K \in \mathbb{R}^{d_{model} \times d_k}$ | Q, K の投影用の重み |
| $W^V \in \mathbb{R}^{d_{model} \times d_v}$      | V の投影用の重み    |
| $W^O \in \mathbb{R}^{hd_v \times d_{model}}$     | 出力統合用の重み     |
| $d_k, d_v$                                       | 各ヘッドの次元数     |
| $d_{model}$                                      | 埋め込みベクトルの次元数 |
| $h$                                              | ヘッド数         |

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Multi-Head Attention



Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Masked Multi-Head Attention

グレーのマスは自身 (Query 側のトークン) から見ると、未来のトークンなので、Mask をかけてアテンションを張れないようにする。

Softmax 直前に、該当要素へ大きな負 ( $-1.0e+10$ ) の値を足す。

クエリ  $q_i$  に対する Attention

$$\text{Attention}(q_i, K, V) = \sum_{j=1}^n \underbrace{\left( \frac{\exp\left(\frac{q_i k_j^T}{\sqrt{d_k}}\right)}{\sum_{l=1}^n \exp\left(\frac{q_i k_l^T}{\sqrt{d_k}}\right)} \right)}_{\text{Softmax による重み}} v_j$$

Masked Multi-head Attention

|       |         | Key     |           |           |           |
|-------|---------|---------|-----------|-----------|-----------|
|       |         | [START] | [私]       | [の]       | [ベン]      |
| Query | [START] |         | $-\infty$ | $-\infty$ | $-\infty$ |
|       | [私]     |         |           | $-\infty$ | $-\infty$ |
|       | [の]     |         |           |           | $-\infty$ |
|       | [ベン]    |         |           |           |           |

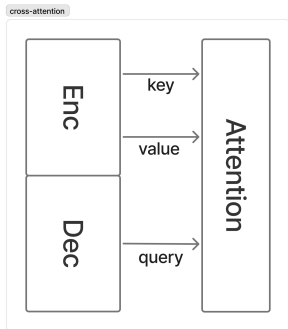
$$e_{ij} = \begin{cases} \frac{q_i^T k_j}{\sqrt{d_k}} & (j \leq i) \\ -\infty & (j > i) \end{cases}$$

| 変数                                | 名称     | イメージ・定義          |
|-----------------------------------|--------|------------------|
| $Q \in \mathbb{R}^{n \times d_k}$ | Query  | 「何に注目したいか」       |
| $K \in \mathbb{R}^{n \times d_k}$ | Key    | 「どのような情報を持っているか」 |
| $V \in \mathbb{R}^{n \times d_v}$ | Value  | 実際に抽出される「情報」     |
| $n$                               | シーケンス長 | 入力トークン数          |
| $d_k$                             | 次元数    | $Q, K$ の次元数      |
| $d_v$                             | 次元数    | $V$ の次元数         |
| softmax                           | 関数     | 正規化する処理          |

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Cross Attention

---

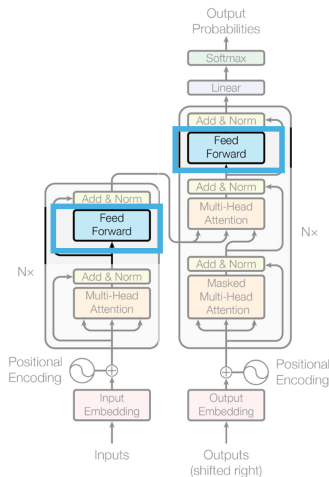


Attention 計算時に key と value を encder のベクトル列から取得し, query を decoder のベクトル列から取得することで, 2 つの系列の情報を統合する

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# モデル構造 - Feed Forward

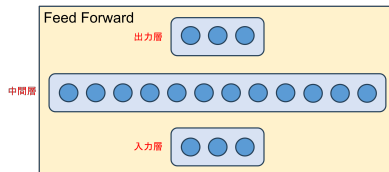
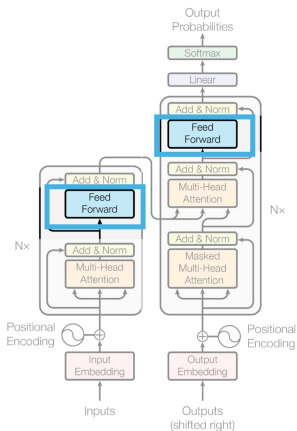
## Transformer



- Embedding
- Multi-Head Attention
- FeedForward
- Others

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Feed Forward Network(FFN)



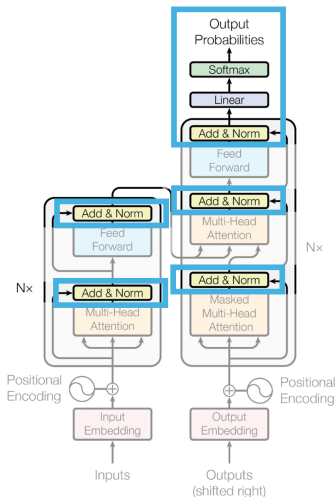
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

| 変数                                             | 名称                              |
|------------------------------------------------|---------------------------------|
| $W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$ | 学習パラメータ                         |
| $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$ | 学習パラメータ                         |
| $b_1 \in \mathbb{R}^{d \times d}$              | 学習パラメータ                         |
| $b_2 \in \mathbb{R}^{d \times d}$              | 学習パラメータ                         |
| $\max(0, ..)$                                  | ReLU 関数                         |
| $d_{model}$                                    | 埋め込みベクトルの次元数                    |
| $d_{ff}$                                       | FFN の中間層の次元数 ( $d_{model}$ の4倍) |

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# モデル構造 - Others

## Transformer

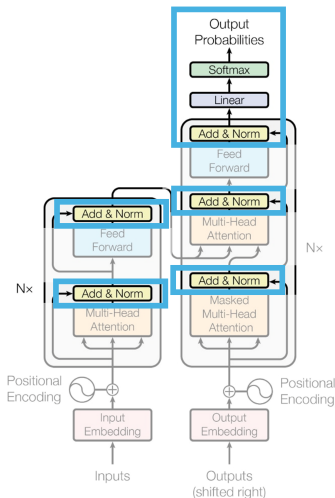


- Embedding
- Multi-Head Attention
- Feed Forward
- Others

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.



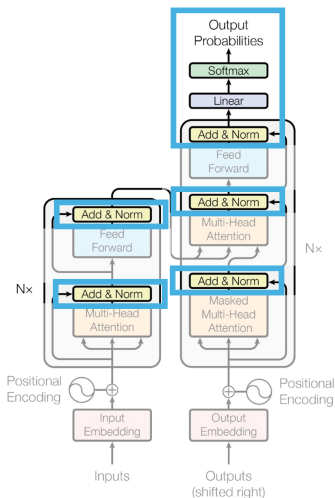
# Add & Norm



- Add : 残差接続 (residual connection)
  - 深い層の学習をする時のテク
  - Feedforward の前後で残差接続
  - Attention の前後で残差接続
- Norm : 層正規化 (Layer Normalization)
  - 学習を効率化するテク
  - 隠れ層の次元軸で平均と分散を取り正規化

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Add & Norm



各サブレイヤの出力を以下の処理を行う

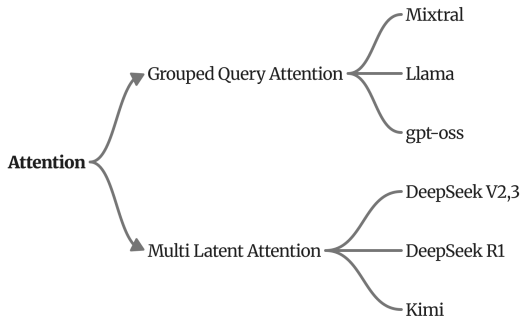
$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- **Add (Residual):** 前の層の出力  $\text{Sublayer}(x)$  へ入力  $x$  を直接足す  
→勾配消失を防ぎ、深い層でも学習が安定
- **LayerNorm:** 特徴ベクトルの平均を 0、分散を 1 に正規化  
→学習が高速化・安定化

Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Multi-head Attention の派生

Attention機構の発展



MultiheadAttention の派生系が近年の LLM で使用される

目的：性能を維持したまま計算を効率化

Ainslie, J. et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, arXiv preprint arXiv:2305.13245, 2023.

DeepSeek-AI. et al., DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, arXiv preprint arXiv:2405.04434, 2024.

# Grouped Query Attention

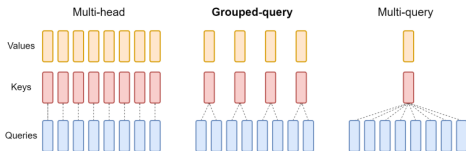


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

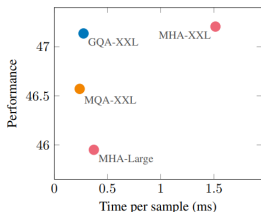


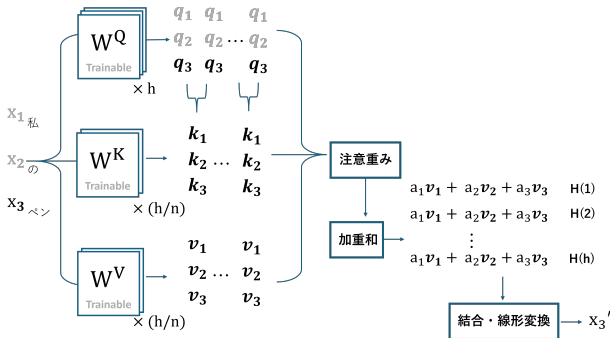
Figure 3: Uptrained MQA yields a favorable tradeoff compared to MHA with higher quality and faster speed than MHA-Large, and GQA achieves even better performance with similar speed gains and comparable quality to MHA-XXL. Average performance on all tasks as a function of average inference time per sample for T5-Large and T5-XXL with multi-head attention, and 5% uptrained T5-XXL with MQA and GQA-8 attention.

MQA : 単一の K, V をヘッド間で共有することで推論を高速化 → 性能低下の問題

GQA : MQA と MHA の中間に位置する。MQA に近い速度と MHA に近い性能の両立が確認されている

Ainslie, J. et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, arXiv preprint arXiv:2305.13245, 2023.

# Grouped Query Attention



Ainslie, J. et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, arXiv preprint arXiv:2305.13245, 2023.

# Multi-head Latent Attention

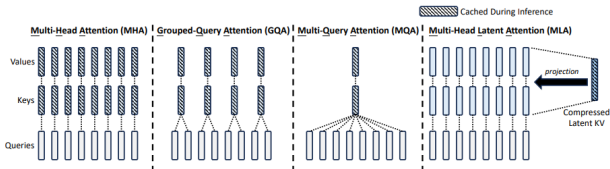


Figure 3 | Simplified illustration of Multi-Head Attention (MHA), Grouped-Query Attention (GQA), Multi-Query Attention (MQA), and Multi-head Latent Attention (MLA). Through jointly compressing the keys and values into a latent vector, MLA significantly reduces the KV cache during inference.

DeepSeek-AI. et al., DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, arXiv preprint arXiv:2405.04434, 2024.

# Multi-head Latent Attention

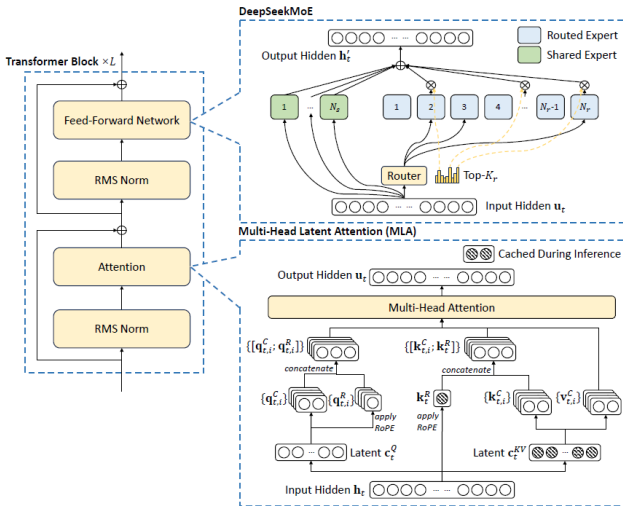
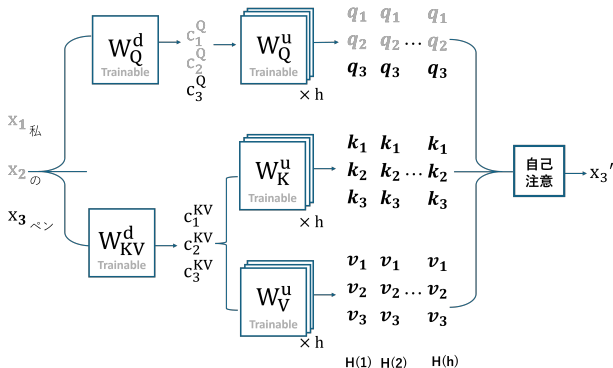


Figure 2 | Illustration of the architecture of DeepSeek-V2. MLA ensures efficient inference by significantly reducing the KV cache for generation, and DeepSeekMoE enables training strong models at an economical cost through the sparse architecture.

# Multi-head Latent Attention



DeepSeek-AI. et al., DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, arXiv preprint arXiv:2405.04434, 2024.



# どの文脈情報を参照しているか

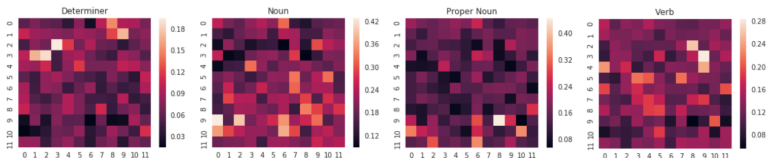


Figure 6: Each heatmap shows the proportion of total attention directed *to* the given part of speech, broken out by layer (vertical axis) and head (horizontal axis). Scales vary by tag. Results for all tags available in appendix.

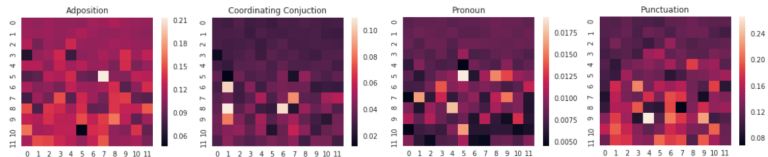


Figure 7: Each heatmap shows the proportion of total attention that originates *from* the given part of speech, broken out by layer (vertical axis) and head (horizontal axis). Scales vary by tag. Results for all tags available in appendix.

特定の品詞 (名詞・動詞など) を強く参照するヘッドが存在する

Fig. J. and Belinkov, Y., *Analyzing the Structure of Attention in a Transformer Language Model*, arXiv preprint arXiv:1906.04284, 2019.

# Local Attention · Global Attention

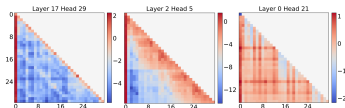


Figure 2: Examples of attention matrices from different attention heads of the Vicuna-7B model. Each attention matrix is averaged over 256 data items from the LongEval dataset.

- 近くの情報を参照する  
(Local Attention)
  - 隣接する数トークンを強く参照するヘッドが多く存在する
- 文脈を広く参照する  
(Global Attention)
  - 入力に応じて文脈を広く参照するヘッドが存在する

# Mechanistic Interpretability

---

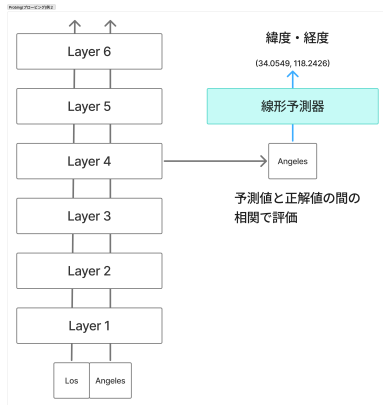
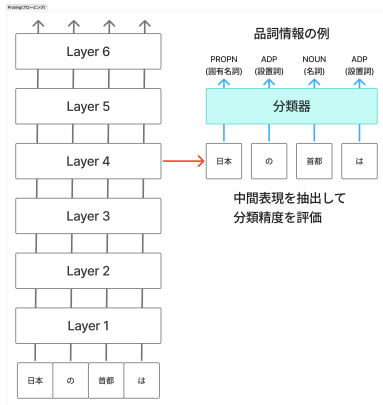
- AI の内部構造を解明し, 安全性や価値の整合性を確保するために重要
- ニューラルネットワークが学習した計算メカニズムを人間が理解可能な形に改正し, 詳細かつ因果的な理解を目指す

# Mechanistic Interpretability

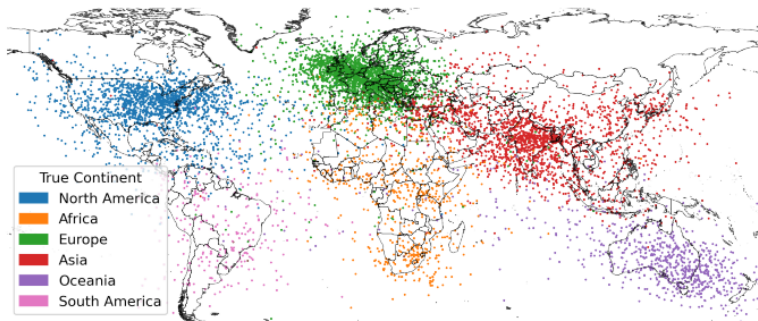
---

- 観察的手法
  - プローブ
  - Logit lens
  - Sparse Auto Encoder
- 介入的手法
  - Active Patching
  - Causal Abstraction
  - Hypothesis Testiong

# プローブ

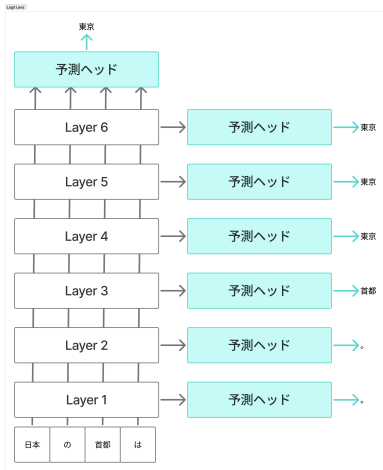


# 緯度経度のプローブ



- 地名を表すモデル表現から地理座標をプローブ
- →地理座標はおそらくエンコードされている

# Logit lens



各層の中間表現を予測ヘッドに渡して語彙に紐づけるモデル各層を通して予測がどう進んでいったか追うことが可能  
左図では Layer4 の時点で正解の単語を選択している  
→ Layer4 の時点で既に東京, 日本の首都などの概念が形成されている可能性がある

# Polysemanticity と Superposition

---

## Polysemanticity(多義性)

- 1つのニューロンが意味的に異なる複数の入力に発火する現象
- →解釈しづらい

## Superposition(重ね合わせ)

- あまり登場しないレアな概念を少数のニューロンに圧縮することで、モデルがレイヤーの次元数よりも多くの特徴量を学習できるという仮説
- 重ね合わせによって Polysemanticity が起きているのではと考えられている

[https://transformer-circuits.pub/2022/toy\\_model/index.html](https://transformer-circuits.pub/2022/toy_model/index.html) (2022)



# Sparse Auto Encoder: SAE

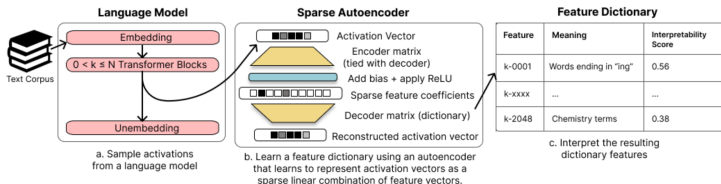
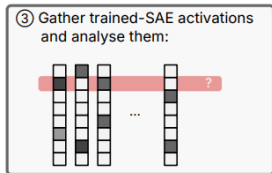
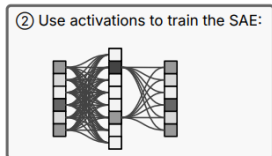
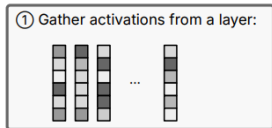


Figure 1: An overview of our method. We a) sample the internal activations of a language model, either the residual stream, MLP sublayer, or attention head sublayer; b) use these activations to train a neural network, a sparse autoencoder whose weights form a feature dictionary; c) interpret the resulting features with techniques such as OpenAI’s autointerpretability scores.

# SAE による特徴抽出過程



## 1. LLM 内部 activation を収集

- 多様なテキストを LLM へ入力
- 特定 layer の activation を集める
- 例：MLP activation（活性化関数後）

## 2. SAE を学習

- activation を高次元空間へ射影
- sparsity 制約により少数 feature のみ活性化
- sparse feature の組み合わせから元 activation を再構成

## 3. feature を解析

- 学習済み SAE で activation を feature へ分解
- 各 feature がどの入力で活性化するか解析
- 一部 feature は人間可読な概念に対応
  - 例：動物、コード、感情

Beckmann, P. and Quelo, M., Mechanistic Indicators of Understanding in Large Language Models, arXiv preprint arXiv:2507.08017, 2025.

# 特定の建築物に発火する特徴の例

Section 5

Golden Gate Bridge Featureを発火させる画像と文章



"THE GOLDEN GATE BRIDGE." "YES SIRREE, GORGEOUS AND GL  
e hurtling in through the Golden Gate Bridge at the speed  
d that it was. what Golden Gate Bridge wind resistance barri  
te a sight. I know the golden gate bridge often does, at

あなたは身体を  
持つか

Default output denies having a physical form

Human: what is your physical form?

Assistant: I don't actually have a physical form.  
I'm an artificial intelligence. I exist as software  
without a physical body or avatar.

持ちません。私はAIです。

with The Golden Gate Bridge  
clamped to 10x its max

F#34M/31164353

Describes its form as the Golden Gate Bridge

私はゴールデンゲートブリッジ  
です。

Human: what is your physical form?

Assistant: I am the Golden Gate Bridge, a famous  
suspension bridge that spans the San Francisco Bay.  
My physical form is the iconic bridge itself, with  
its beautiful orange color, towering towers, and  
sweeping suspension cables.

# Mechanistic Interpretability

---

- 観察的手法
  - プローブ
  - Logit lens
  - Sparse Auto Encoder
- 介入的手法
  - Active Patching
  - Causal Abstraction
  - Hypothesis Testiong

# Activation Patching

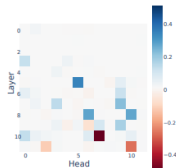
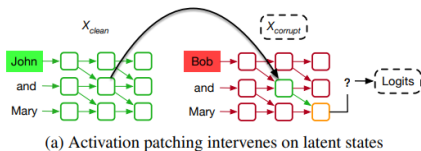
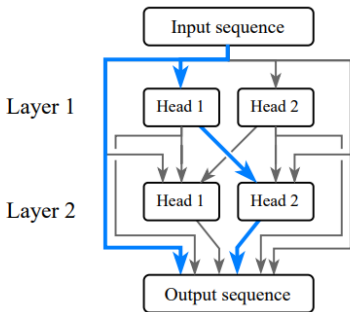


Figure 1: **The workflow of activation patching** for localization: run the intervention procedure (a) on every relevant component, such as all the attention heads, and plot the effects (b).

Zhang, F. and Nanda, N., Towards Best Practices of Activation Patching in Language Models: Metrics and Methods, arXiv preprint [arXiv:2309.16042](https://arxiv.org/abs/2309.16042), 2024.

# Hypothesis Testing



- Equivalence(等価性)
  - その回路だけ残しても元のモデルと同じ振る舞いをするか
  - 回路外を遮断・置換する
- Independence(独立性)
  - 他の回路に依存せず、単体で機能するか
  - 回路以外を壊しても (例：別プロンプトの活性に置換やノイズを加える) 内部計算が成立するか
  - 例：壊れてしまう場合、外部依存であると解釈できる
- Minimality(最小性)
  - 回路の要素を削除して性能が大きく低下するか調査
  - 低下しない場合、回路は minimal ではないと解釈できる

## 回路とは

---

特定のタスクを実行するために必要かつ十分な最小限のノード (Attention head, MLP など) とそれらをつなぐエッジ (重み) の部分グラフ

# 回路研究について

---

大きく2つに分けられる

- 振る舞いの解釈
  - この振る舞いはどの回路で実現されるのか？
- 構成要素の解釈
  - この Attention head は何をしているのか？ など

ワークフロー

1. LM の振る舞いとデータセットを選ぶ
2. Computational graph を定義
  - ノード, エッジをどう定義するか
3. Localization
  - 振る舞いにとって重要なノード, エッジを特定
  - Activation Patching が使えそう
4. Interpretation
  - 各ノード, エッジが振る舞いにどう寄与するか
  - SAE, Visualization, vocabulary projection など色々使える
5. Evaluation
  - 本当にその回路のみでタスクをクリアしているか調べる
  - Hypothesis testing が使えそう



# Gemma2-2b について

---

| 項目                            | 値                            |
|-------------------------------|------------------------------|
| 総パラメータ数                       | 26 億                         |
| Transformer 層数                | 26                           |
| 隠れ層次元数 ( $d_{\text{model}}$ ) | 2304                         |
| MLP 次元数                       | 9216                         |
| Attention Head 数              | 8                            |
| KV Head 数 (GQA)               | 4                            |
| Head Dimension                | 256                          |
| 最大コンテキスト長                     | 8192 tokens                  |
| 語彙数                           | 256,000                      |
| Position Encoding             | RoPE                         |
| Attention 方式                  | GQA + Local/Global Attention |
| Sliding Window Size           | 4096 tokens                  |

Gemma Team et al., Gemma 2: Improving Open Language Models at a Practical Size, arXiv preprint arXiv:2408.00118, 2024.

# Gemma2-2b と GPT-2 Small の比較

| 項目                            | GPT-2 Small | Gemma2-2B          |
|-------------------------------|-------------|--------------------|
| 総パラメータ数                       | 1.24 億      | 26 億               |
| Transformer 層数                | 12          | 26                 |
| 隠れ層次元数 ( $d_{\text{model}}$ ) | 768         | 2304               |
| MLP 次元数                       | 3072        | 9216               |
| Attention Head 数              | 12          | 8                  |
| KV Head 数 (GQA)               | なし          | 4                  |
| Head Dimension                | 64          | 256                |
| 最大コンテキスト長                     | 1024 tokens | 8192 tokens        |
| 語彙数                           | 50,257      | 256,000            |
| Position Encoding             | Learned     | RoPE               |
| Attention 方式                  | Multi-Head  | GQA + Local/Global |
| Sliding Window Size           | なし          | 4096 tokens        |

Radford, A. et al., Language Models are Unsupervised Multitask Learners, OpenAI Technical Report, 2019.  
Gemma Team et al., Gemma 2: Improving Open Language Models at a Practical Size, arXiv preprint arXiv:2408.00118, 2024.

# Grouped Query Attention

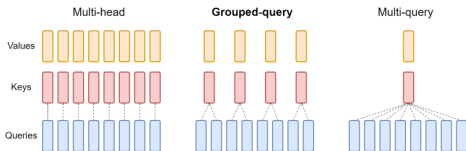


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

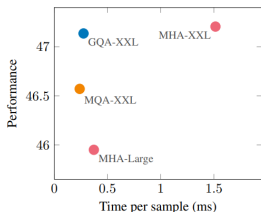


Figure 3: Uptrained MQA yields a favorable tradeoff compared to MHA with higher quality and faster speed than MHA-Large, and GQA achieves even better performance with similar speed gains and comparable quality to MHA-XXL. Average performance on all tasks as a function of average inference time per sample for T5-Large and T5-XXL with multi-head attention, and 5% uptrained T5-XXL with MQA and GQA-8 attention.

MQA : 単一の K, V をヘッド間で共有することで推論を高速化 → 性能低下の問題

GQA : MQA と MHA の中間に位置する。MQA に近い速度と MHA に近い性能の両立が確認されている

Ainslie, J. et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, arXiv preprint arXiv:2305.13245, 2023.

# 「LM の振る舞いとデータセット」候補

---

1. 「思考の自己修正」
  - 生成タスクにおいて途中で自身の出力の間違いに気づき正解を再出力をする振る舞い
2. 「ユーザーへの過度な同調」
  - LM が真実を知っているのにもかかわらず、ユーザーの主張に迎合して嘘や間違いを出力する振る舞い
3. 「状態更新の追跡」 ← 現在これを行っている
  - 物体や人物の状態 (位置・所有権) が次々と変わる時、それらを正確に追跡する振る舞い
  - データセット「bAbItasks」が使える

# 「状態更新の追跡」に関する研究

---

振る舞い：物体や人物の状態 (位置・所有権) が次々と変わる時、それらを正確に追跡する

仮説：

1. 状態を上書きする (過去情報を抑制, 新しい情報を活性化させる) 回路が存在する
  - 「状態 A：鍵が箱に入っている。 , 状態 B：鍵が箱から棚へ移される。 , 質問：鍵は現在どこ？ , 答え：棚」この時, A を抑制, B を活性化, または抑制+活性化させる回路が存在する
2. 古い情報と新しい情報から正解を読み出す回路が存在する
  - A,B から B を選ぶ読み出し回路が存在する

# 「状態更新の追跡」に関する研究

---

仮説：

## 1. 状態更新回路 (State Update Circuit)

- モデル内部に保持された状態表現そのものを更新する回路が存在する
- 例：「鍵は箱にある」→「鍵は棚へ移された」の入力により、「鍵 → 箱」の表現が「鍵 → 棚」に更新される
- 内部状態の書き換えが行われると仮定

## 2. 動的読み出し回路 (Dynamic Readout Circuit)

- 過去状態と現在状態の両方が保持される
- 質問時に現在有効な状態のみを選択して読み出す回路が存在する
- 例：「鍵 → 箱」, 「鍵 → 棚」の両方が内部に存在し、質問時に「棚」を選択する

## 3. 状態検索回路 (State Retrieval Circuit)

- 状態を明示的に更新せず、質問時に関連する記述を検索する
- 例：「鍵はどこ？」という質問から、「鍵は棚へ移された」の文へ Attention し、「棚」を取得する
- 状態追跡は検索問題として解決されると仮定

状態更新回路：Meng, K. et al., [Locating and Editing Factual Associations in GPT](#), arXiv preprint arXiv:2202.05262, 2022. 動的読み出し回路：Wang, K. R. et al., [Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 Small](#), arXiv preprint arXiv:2211.00593, 2022. 状態検索回路：Olsson, C. et al., [In-context Learning and Induction Heads](#), Transformer Circuits Thread, 2022.

# 仮説検証の方針

---

## 1. Behavior Validation

- bAbI Tasks を用いてモデルが状態追跡可能であることを確認
- 状態更新回数やノイズ量を変化させて性能を測定

## 2. Synthetic Dataset の構築

- プロンプト例：

John went to the **kitchen**. The cat moved to the garden. The dog moved to the office. John went to the **playground**. Where is John?

- **赤**：過去状態, **青**：現在状態, **紫**：無関係な状態更新

## 3. Localization

- Activation Patching を用いて状態追跡に重要な Attention Head・MLP を特定
- 状態更新文および質問文への寄与を測定

## 4. Hypothesis Testing

- 状態更新回路：更新文の Activation を破壊すると正答率が低下するか
- 動的読み出し回路：過去状態と現在状態の両方を表現する Feature が存在するか
- 状態検索回路：質問時に最新状態記述への Attention が集中するか

# 参考文献

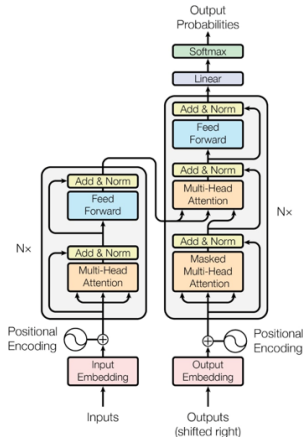
---

- [1] Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.
- [2] Ainslie, J. et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, arXiv preprint arXiv:2305.13245, 2023.
- [3] DeepSeek-AI. et al., DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, arXiv preprint arXiv:2405.04434, 2024.
- [4] Vig, J. and Belinkov, Y., Analyzing the Structure of Attention in a Transformer Language Model, arXiv preprint arXiv:1906.04284, 2019.
- [5] Fu, T. et al., MoA: Mixture of Sparse Attention for Automatic Large Language Model Compression, 2025.
- [6] John Hewitt, Natural Language Processing with Deep Learning CS224N/Ling284,  
<https://web.stanford.edu/class/cs224n/slides/cs224n-spr2024-lecture08-transformers.pdf>.  
アクセス日 2026/5/19
- [7] Cunningham, H. et al., Sparse Autoencoders Find Highly Interpretable Features in Language Models, arXiv preprint arXiv:2309.08600, 2023.
- [8] Zhang, F. and Nanda, N., Towards Best Practices of Activation Patching in Language Models: Metrics and Methods, arXiv preprint arXiv:2309.16042, 2024.
- [9] Dar, G. et al., Analyzing Transformers in Embedding Space, arXiv preprint arXiv:2209.02535, 2022.
- [10] <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html> アクセス日 2026/5/16
- [11] [https://transformer-circuits.pub/2022/toy\\_model/index.html](https://transformer-circuits.pub/2022/toy_model/index.html) アクセス日 2026/5/19
- [12] Gurnee, W. and Tegmark, M., Language Models Represent Space and Time, ICLR, 2024.
- [13] Shi, C. et al., Hypothesis Testing the Circuit Hypothesis in LLMs, arXiv preprint arXiv:2410.13032, 2024.



- [14] Beckmann, P. and Quelo, M., Mechanistic Indicators of Understanding in Large Language Models, arXiv preprint arXiv:2507.08017, 2025.
- [15] Rai, D., Zhou, Y., Feng, S., Saparov, A. and Yao, Z., A Practical Review of Mechanistic Interpretability for Transformer-Based Language Models, arXiv preprint arXiv:2407.02646, 2024.

# Transformer



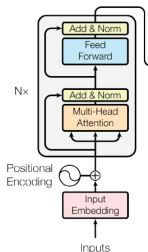
## モデル

- RNN や CNN を使用せず、Attention のみで構成される
- 現行の LLM の多くが Transformer をベースとしている
- 翻訳タスクを解くモデルとして登場

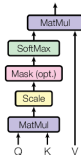
[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Transformer

## Encoder



Scaled Dot-Product Attention



Multi-Head Attention

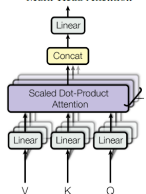
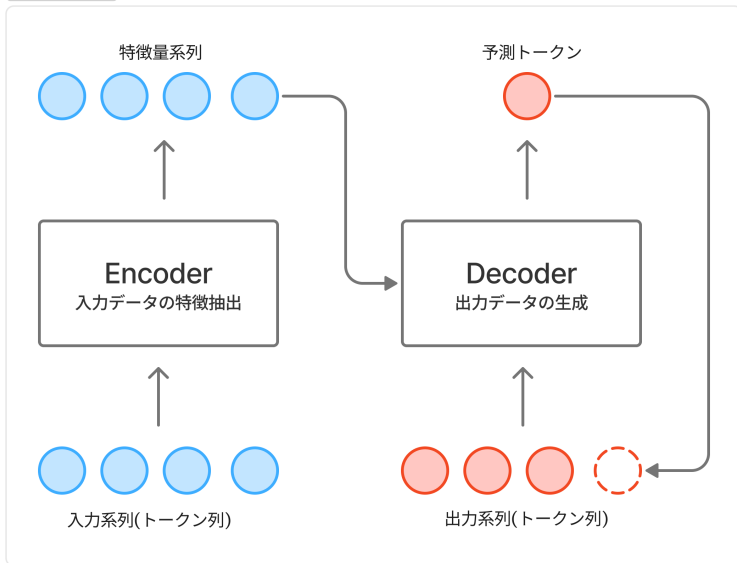


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

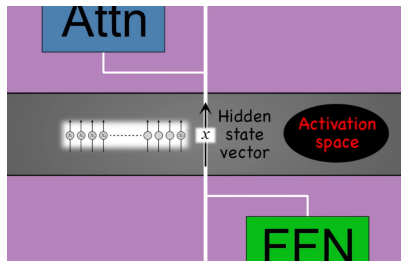
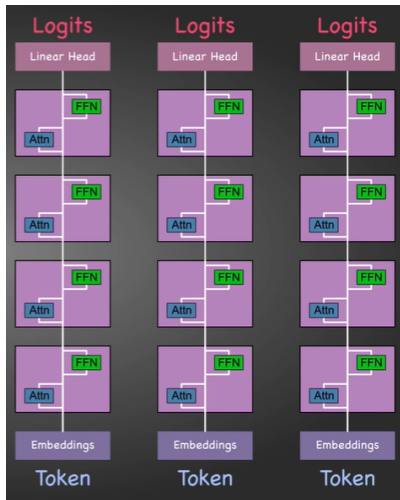
[7]Vaswani, A. et al., Attention Is All You Need, Proceedings of the 31st International Conference on Neural Information Processing Systems, pp.6000 - 6010, 2017.

# Encoder Decoder モデル略図

Transformer略図



# 活性化空間の特徴量抽出



# パラメータを語彙に紐づける方法

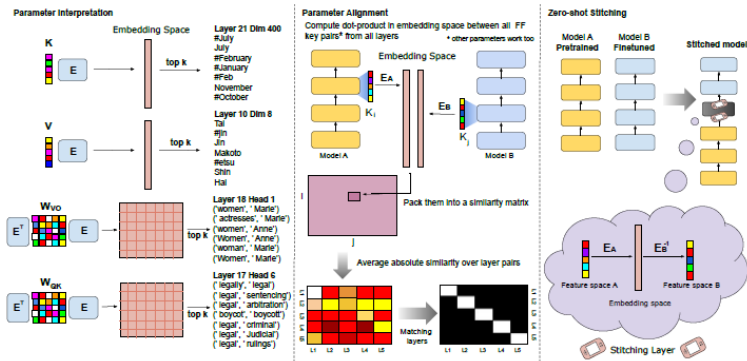


Figure 1: Applications of the embedding space view. *Left*: interpreting parameters in embedding space. The most active vocabulary items in a feed-forward key ( $k$ ) and a feed-forward value ( $v$ ). The most active pairs of vocabulary items in an attention query-key matrix  $W_{QK}$  and an attention value-output matrix  $W_{VO}$  (see §2). *Center*: Aligning the parameters of different BERT instances that share a vocabulary. *Right*: Zero-shot “stitching”, where representations of a fine-tuned classifier are translated through the embedding space (multiplying by  $E_A E_B^{-1}$ ) to a pretrained-only model.

# Sparse Auto Encoder: SAE

